Fire Detection & Autonomous Response at the Edge

D7065E Embedded Intelligence at the Edge, Final Report

Group 3: Binay Mishra

Contents

1. Summary **2**

2. Use case and building context **2**

3. Requirements **3**

4. Architecture **4**
4.1 Context diagram (C4 Level 1) 4
4.2 Container diagram (C4 Level 2) 5
4.3 Component diagram (C4 Level 3) 6
4.4 Design decisions and alternatives 8

5. Interfaces and data contracts **10**

6. Simulating the sensor values (the physical model) **11**

7. Autonomous services and data pipeline **11**
7.1 The autonomous services 11
7.2 The data pipeline 12

8. Behaviour **13**
8.1 Key scenario 13
8.2 Door lifecycle (state diagram) 16

9. Deployment and component placement **17**

10. Test plan **18**

11. Evaluation and results **19**

12. Dashboard **22**

13. Risks and critical reflection **23**

1. Summary

This system detects fires in the A-building at LTU and responds autonomously. It is built in two tiers. **Tier 1** is an edge controller per floor, running on a Raspberry Pi: it reads every room's smoke, CO and temperature, raises a confirmed alert on a deterministic multi-sensor rule and an earlier possible alert when an isolation forest flags the room, and releases its own floor's fire doors within one 5 s tick, with no dependency on a server, a broker or a language model. **Tier 2** is a server agent: it asks a self-hosted LLM for a verdict on every edge alert, coordinates the other floors, computes evacuation routes, drives the 3D viewer, and stands the building down when readings are clear. A standby controller on a second node takes over a floor within 30 s if its active controller dies; the server takes over a floor if both controllers are gone; and the whole response still works when the LLM is unreachable.

BuildSim provides the building: 950 rooms on three floors, 124 fire doors, sprinklers and the 3D view. This project provides the physics, the sensor degradation, the two tiers of intelligence and the response around it. Life-safety actions happen at the edge, judgement at the server, and every layer degrades gracefully.

Measured on the project cluster (k3s; two Raspberry Pi edge nodes and an amd64 server node) during the acceptance run: ignition to own-floor doors released in 14.7 s; isolation-forest inference p50 17.8 ms on the Pi; LLM verdict p50 2.1 s, p95 3.2 s; verdict to cross-floor release 25 ms; standby promotion 30 s after controller loss. Nine acceptance scenarios pass, including controller failover mid-incident, server takeover of a floor with no controller, LLM outage, broker restart and cold start.

Field	Value
Project title	Fire Detection & Autonomous Response at the Edge
Team members	Binay Mishra (Group 3)
Use case	Autonomous fire detection, floor-local door release, building-wide coordination and evacuation
Repository	https://github.com/applied-ai-ltu/d7065e
Version / date	v2.0 / 2026-10-12

Changes since the proposal. Two design changes were made because measurements demanded them. A third was a deliberate choice: the proposal demonstrated on one room, and the sensor gateway was parameterised by floor so that the full building of 950 rooms is covered.

1. **From one agent to two tiers.** The proposal had a single server agent polling BuildSim and deciding everything. The LLM was on the critical path. Measured LLM latency of 5–7 s per plan, and the requirement that door release must not depend on a server, led to a per-floor edge controller that acts first and reports second. The LLM is no longer on the life-safety path; it confirms, rejects and coordinates.
2. **The LLM returns a verdict, not a plan.** The proposal had the LLM generate the response plan (sprinklers, doors, routes). The response is now derived deterministically from a verdict `{is_fire, confidence}`; sprinklers are mechanical in the physics model and are never commanded. A shorter answer is faster to produce: verdict latency fell from 5.2 s to 2.1 s at p50. It is also safer, because the LLM can no longer produce a plan that opens the wrong doors or routes people through the fire; the plan is computed from the verdict by code that is tested.

2. Use case and building context

The building. The A-building at LTU is modelled in BuildSim on three floors: level0 (318 rooms, 41 fire doors), level1 (351 rooms, 46 fire doors) and level2 (287 rooms, 37 fire doors). BuildSim

holds the building model, exposes a REST API for reading and writing equipment state, provides a navigation graph per floor with stairwell edges between floors, and streams live state to a 3D web viewer. Everything in this project sits behind the BuildSim API.

What talks to BuildSim. Three components touch building state:

- The **physics engine and sensor gateway**, inside the server process, read the floor plans, door graph and equipment list at start-up, advance the room physics, degrade the readings with sensor faults, and publish them over MQTT.
- Each **edge controller** discovers its floor's fire doors from the equipment list and sets their state directly (`PUT /api/actuators/{id}/state`). This is the only life-safety write path, and it does not pass through the server.
- The **server agent** pushes visualisation (effects, alert cards, coverage) and sets a floor's doors itself only when that floor has no controller left.

Actors. The building operator watches the 3D viewer and the Grafana dashboards, and can start or stop test fires, trigger false alarms and clear the viewer through the control API. The operator is not part of the response: detection, door release, confirmation and stand-down run without any human input.

A typical incident. A fire starts in A109 on level0. Smoke and CO rise and spread through the door graph to A108, A110 and the corridor. Within three ticks, the level0 controller sees smoke above 10 %obs/m and CO above 20 ppm, releases all 41 fire doors on its floor and alerts the server. The server asks the LLM, gets a confirmed verdict about two seconds later, releases the doors on level1 and level2, publishes evacuation routes and paints the viewer. Once the fire is out and readings have been clear for 60 s, the server stands the building down and re-locks every floor. The same incident is shown with measured timestamps in Figure 6.

Requirements. Three requirements follow from the use case. Detection and door release must be fast and must not depend on anything outside the floor, so they run on the edge controller. A confirmed fire needs more context than a threshold gives, so the server reviews every alert with an LLM before the rest of the building reacts. And each component must keep working, in a defined way, when the components around it are unavailable.

3. Requirements

Each requirement has an ID, an acceptance criterion and the test that verifies it (Section 10). FR are functional requirements, NFR are non-functional, and SAF are the safety requirements.

ID	Type	Requirement	Pri.	Acceptance criterion	Verified by
FR-01	Func.	Detect a fire within 60 s of ignition	Must	Edge fire decision logged within 60 s	T-06
FR-02	Func.	Distinguish a real fire from a smoke-only false alarm	Must	Smoke-only event rejected by Tier 2, no door command	T-05, T-07
FR-03	Func.	Release the fire doors of the affected floor without any server involvement	Must	Door release issued in the same tick as the fire decision, server offline or not	T-06, T-02
FR-04	Func.	Compute evacuation routes that avoid affected rooms	Must	Route published on confirm; affected rooms not on the path	T-06
FR-05	Func.	Propagate fire to physically adjacent rooms only	Must	Spread follows the door graph and a 30-unit wall radius; no spread across courtyards	T-06, T-U
FR-06	Func.	Stand the building down only on evidence	Must	Stand-down after 60 s of clear readings per room; refused while a room reads critical	T-06, T-U

ID	Type	Requirement	Pri.	Acceptance criterion	Verified by
NFR-01	Non-func.	Monitor all 950 rooms on three floors	Must	All rooms present in the database and in each controller's room count	T-04
NFR-02	Non-func.	Survive LLM unavailability without losing detection or response	Must	Fallback confirms within one tick; cross-floor release still issued	T-02
NFR-03	Non-func.	Ingest the full sensor stream	Must	Pipeline sustains $\geq$400 readings/s with an empty queue	T-09
NFR-04	Non-func.	Run detection on the edge device	Must	ONNX detector on the Pi, p95 $<$ 100 ms per room	T-06, T-09
NFR-05	Non-func.	No secrets or Vault tokens in the repository or the pods	Must	Repository scan finds no credentials; server reads secrets from a Kubernetes Secret, edge controllers need none	T-10
NFR-06	Non-func.	Survive a broker restart	Should	Edges and server reconnect; fire handled afterwards	T-03
NFR-07	Non-func.	Tolerate a stuck sensor	Should	Frozen sensor cannot block detection or hold the stand-down	T-03, T-U
SAF-01	Safety	Release fire doors on every floor when a fire is confirmed on any floor	Must	Release commands to all other floors within 1 s of the verdict	T-06
SAF-02	Safety	Keep a floor covered when its active controller dies	Must	Standby promotes within 30 s; doors already released stay released	T-08
SAF-03	Safety	Keep a floor covered when both its controllers are gone	Must	Server takes the floor over within 30 s and actuates its doors directly	T-01
SAF-04	Safety	Doors are never re-locked by a handover, restart or false alarm	Must	Re-lock only on a stand-down with the floor clear, or on ALL CLEAR	T-05, T-07, T-08

Detection speed (FR-01, FR-03) favours low thresholds; false-alarm rejection (FR-02) favours high ones. The two tiers split the decision so that both can be met: the edge releases its own floor's doors on a low threshold, and the server requires an LLM verdict before the rest of the building reacts.

4. Architecture

4.1 Context diagram (C4 Level 1)

The system reads and commands the building only through BuildSim. The operator watches the viewer and the dashboards. The LLM inference service and the observability stack are external, and the system has to keep working when either is unavailable.

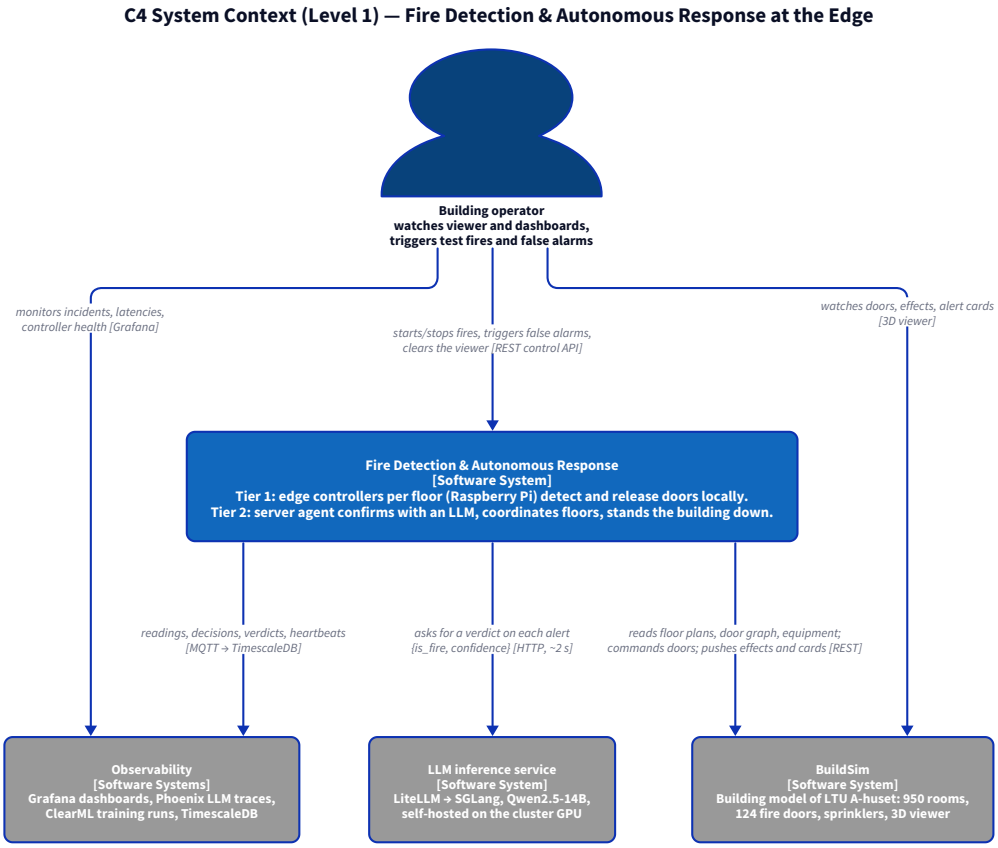


Figure 1: System context. The Raspberry Pi edge controllers are inside the system boundary; the LLM service and observability are outside it and not required for life-safety actions.

4.2 Container diagram (C4 Level 2)

Each box in Figure 2 is a service, drawn on the node where it runs. Tier 1 is six edge controllers: one active per floor on a Raspberry Pi and one standby per floor on an amd64 worker, built from the same image with the same resource spec. Tier 2 is the server process, which hosts the physics engine and sensor gateway, the agent, the MQTT-to-database pipeline and the control API. MQTT connects the tiers: readings go down to the controllers; alerts, decisions and heartbeats go up to the server; commands go down again. Both tiers reach BuildSim by REST; only Tier 2 reaches the LLM.

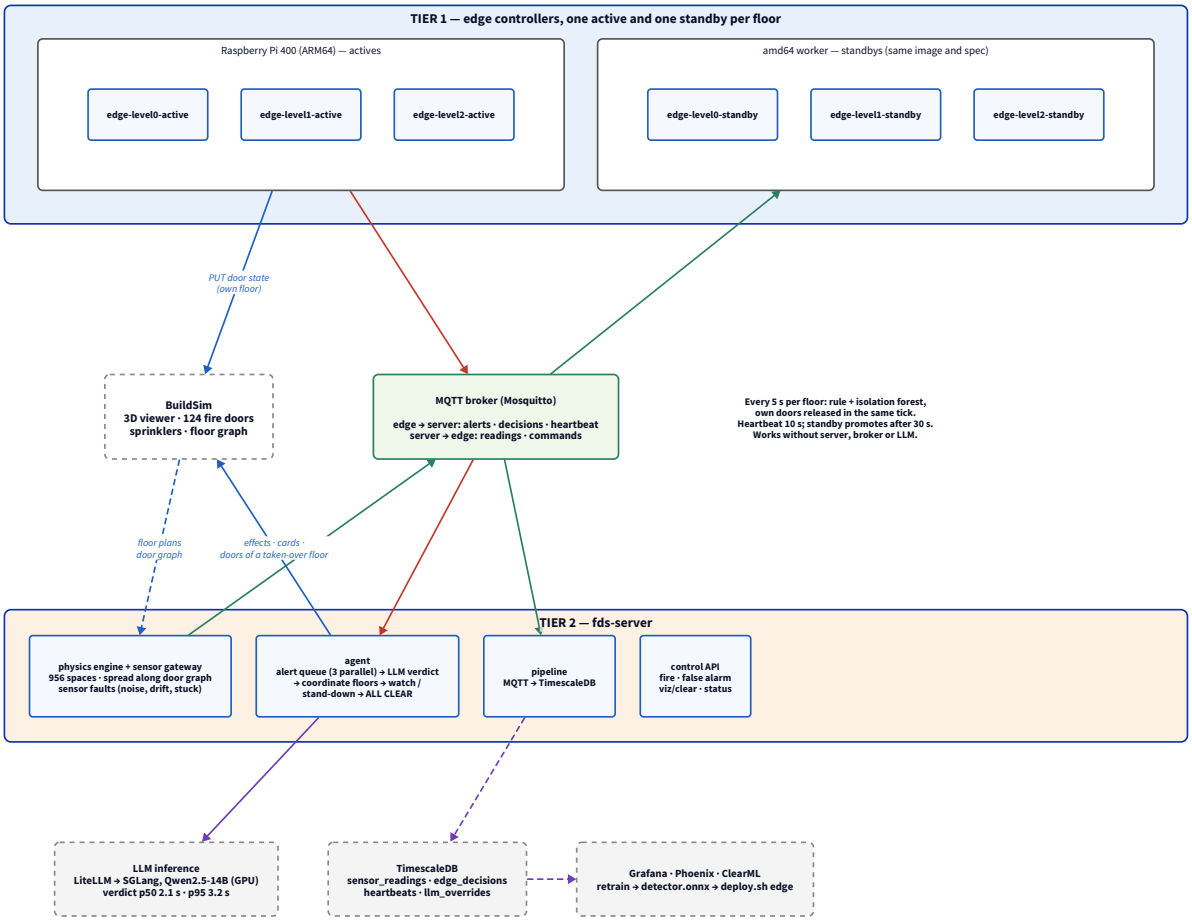


Figure 2: Containers and their placement on the k3s cluster. Red edges carry alerts and heartbeats; green edges carry readings and commands; blue edges are REST calls to BuildSim; purple edges are data and model flows.

The control loop is: the physics engine advances the rooms and publishes degraded readings; the controller on that floor decides; it writes door state into BuildSim; on the next tick the engine and the viewer reflect the new door state. The server is outside this loop.

4.3 Component diagram (C4 Level 3)

Figures 3 and 4 show the two decision components: the edge controller's per-room tick and the server agent's per-alert handling.

Tier 1, per room every 5 s. A deterministic multi-sensor rule is evaluated first: smoke $\geq$ 10 %obs/m and CO $\geq$ 20 ppm is a fire, and the learned model cannot override it. A second rule covers a CO sensor the controller has marked faulty (value frozen for 60 s): smoke $\geq$ 20 alone then confirms. Below those, the isolation forest handles the ambiguous band: a room with smoke $\geq$ 5 that the model flags as anomalous is sent to the server as `possible`, and a room with smoke $\geq$ 10 is sent even if the model is quiet. On the first `fire` on its floor the controller releases every fire door it owns and alerts the server. Alerts are rate-limited per room (on state change, on a 1.5× rise, or every 60 s) so a long incident does not flood the server.

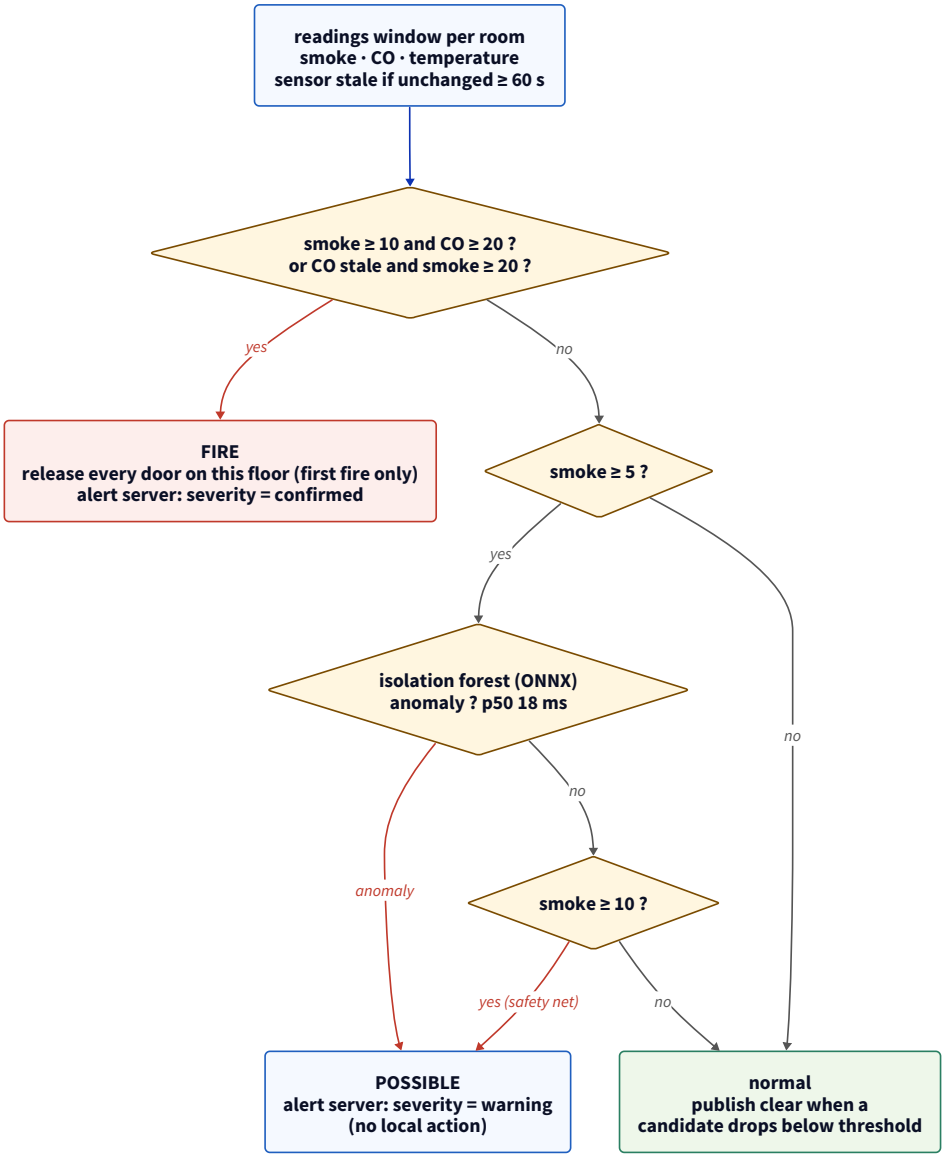


Figure 3: Tier 1: one room, one tick. A `fire` or `possible` outcome is published to `alerts/<floor>/<room>`; red paths act, grey paths defer.

Tier 2, per alert, three in parallel. Alerts are de-duplicated: the same room and severity within 60 s is dropped, and a confirmed room is not re-assessed. The agent assembles what the LLM sees: readings and their rates, the sensors the edge marked stale, the rooms at risk from spread, and the evacuation route. The LLM returns a verdict. Confidence ≥ 0.7 confirms the fire: cross-floor release (once per incident), routes, viewer effects and a red card. Anything lower puts the room on a 60 s watch with an orange card, and the readings decide from there: clear (smoke < 2 and CO < 5 on live sensors) stands the room down, fire-level readings escalate it on the deterministic rule, anything in between keeps it on watch. A confirmed room is stood down after 60 s of clear readings. When no room anywhere is confirmed or on watch, ALL CLEAR re-locks every floor and clears the viewer. If the LLM is unreachable, the fallback applies the same multi-sensor rule with confidence 0.9 or 0.2.

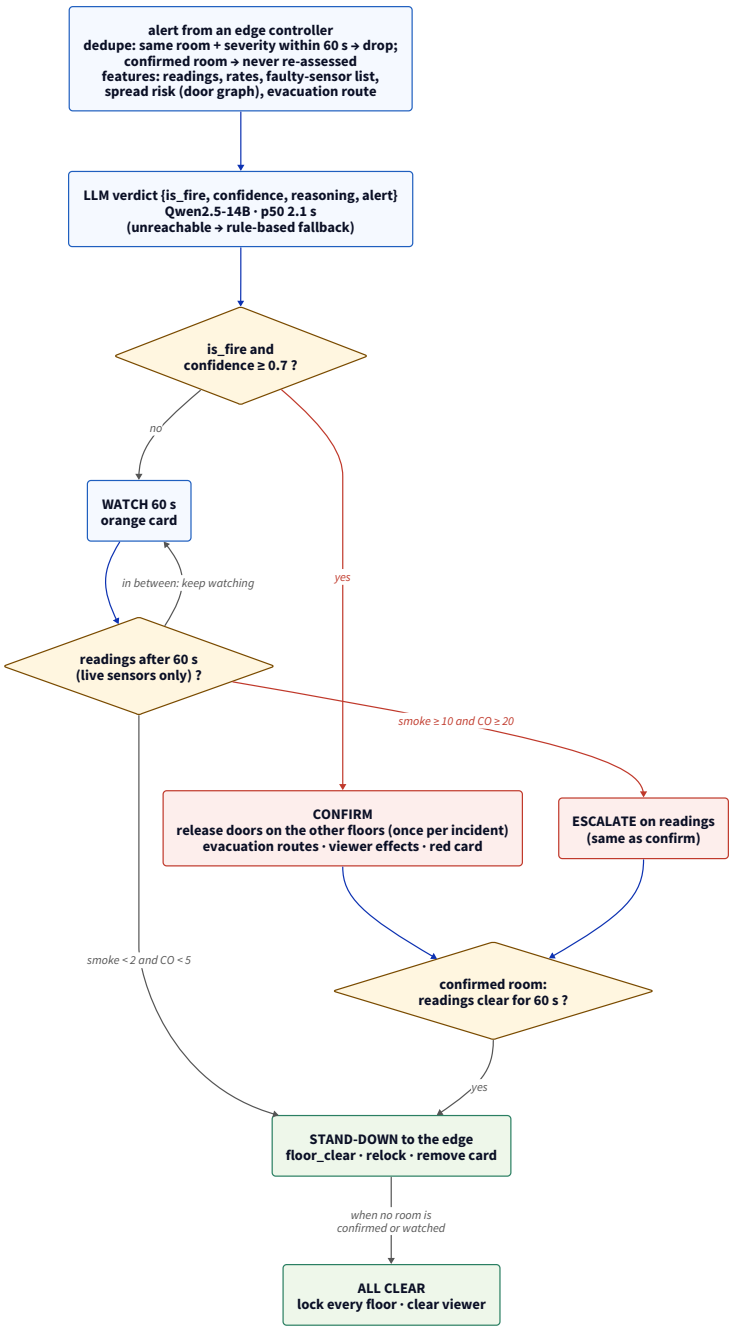


Figure 4: Tier 2: one alert. The stand-down is sent on `commands/<floor>/stand-down`; the edge re-locks only if the floor is clear and it owns the release.

4.4 Design decisions and alternatives

The following table shows the option chosen, the alternative considered, the requirement it serves and the trade-offs.

Decision	Chosen	Alternatives	Justification	Trade-off
Topology	Two tiers: edge per floor + server	Server-only; edge-only	Door release must not depend on a server, a broker or an LLM (FR-03, SAF-01–03). Cross-floor coordination, routes and explanation need a building-wide view.	Two code bases, state split across tiers

Decision	Chosen	Alternatives	Justification	Trade-off
Edge decision	Deterministic rule; isolation forest only on the ambiguous band	Model decides; thresholds only	The rule is auditable and cannot be overridden by a learned model (FR-01). Thresholds alone cannot separate smoke-only events from fire (FR-02), which is what the model is for.	Hand-set thresholds
Edge model	Isolation forest, ONNX Runtime on the Pi	Autoencoder; TFLite; classifier	17.8 ms p50 on a Pi 400, sklearn-native export, parity 1.000, retrainable from real windows (NFR-04). A classifier needs labels, which the system only accumulates from LLM verdicts over time.	Less expressive than a neural model
Tier-2 model	Qwen2.5-14B via LiteLLM/S-GLang, verdict-only contract	Full-plan generation; cloud API; calibrated decision model	The response is derived deterministically from `{is_fire, confidence}`; verdict latency went from 5.2 s to 2.1 s p50; self-hosted, so it works without internet access (NFR-02). Calibrated decision models (Jev and open-weight equivalents) could fit the verdict function better but were released only weeks before the deadline.	Confidence is not calibrated
Alert handling	Bounded concurrency (3) + per-room dedupe	Serial queue	Alert-to-verdict queueing was 24 s p50 under a cluster of alerts; parallel assessment removed it. Dedupe keeps LLM load proportional to incidents, not ticks.	Ordering is per room, not global
Door ownership	Edge owns its floor, server owns cross-floor; stand-down carries floor_clear and relock	Server owns all doors	Survives failover and restart without state transfer: a fresh controller re-locks because the server tells it the floor is clear, not because it remembers opening the doors (SAF-02, SAF-04).	Two release flags per floor to reason about
Failover	Active/ standby per floor, heartbeat 10 s, promote at 30 s; server takeover when both are silent	Single controller; k8s restart only	A pod restart takes about 20 s and loses state; a hot standby on another node keeps detecting throughout (SAF-02). Server takeover covers the case where the node itself is gone (SAF-03).	Standby costs a second pod per floor
Stale sensors	Value frozen $\geq$ 60 s is reported stale; stand-down ignores it, detection falls back to strong smoke alone	Trust every reading	A CO sensor stuck at 5.09 ppm held three floors open; one stuck at 0.00 in the fire room blocked detection (NFR-07). Both occurred during the acceptance run.	A frozen but correct value is treated as faulty
Spread model	BuildSim door graph + 30-unit wall radius	50-unit radius	A plain radius crossed a courtyard (A109 to A323 at 45 units); the door graph alone is a spanning tree and would under-spread the fire (FR-05).	Centroid geometry, not walls
Broker / store	Mosquitto / TimescaleDB	Kafka / InfluxDB	Small footprint, SQL for Grafana.	One more service each
Deployment	k3s, node-local images, `deploy.sh`	ArgoCD GitOps	Images never leave the cluster (`imagePullPolicy: Never`), so GitOps could not deploy them; `selfHeal` would also have reverted the scale-to-zero used in the failover tests.	No declarative sync

Decision	Chosen	Alternatives	Justification	Trade-off
Secrets	Kubernetes Secret fds-app; Vault for tooling	Vault token in the pod	No root token in the namespace (NFR-05); tooling on the development machine keeps using Vault.	Secret created manually once

5. Interfaces and data contracts

Two communication patterns are used. MQTT pub/sub carries everything time-critical between the tiers because one publisher feeds several consumers, the broker buffers short outages, and topic hierarchy gives per-floor routing. REST carries every call to BuildSim and the operator's control API, because each is a one-to-one request whose result the caller must confirm.

Every reading is a JSON object with explicit units:

```
{ "sensor_id": "level0-A109-smoke", "room": "A109",
  "level": "level0", "type": "smoke", "unit": "%obs/m",
  "value": 14.29, "timestamp": "2026-10-04T13:46:07Z" }
```

An edge alert carries the snapshot the controller decided on, including the stale-sensor list and the model's inference time:

```
{ "room": "A109", "floor": "level0", "decision": "fire",
  "severity": "confirmed", "smoke": 14.29, "co": 37.68,
  "temperature": 17.63, "stale": [], "if_anomaly": true,
  "if_ms": 17.49, "node": "pi400-master-01", "role": "active" }
```

Interface	Protocol	Endpoint / topic
Gateway → broker	MQTT	`building/{level}/{room}/{type}`
Edge ← broker	MQTT	`building/{level}/#`, `commands/{level}/#`, `heartbeat/{level}/#`
Edge → broker	MQTT	`alerts/{level}/{room}`, `decisions/{level}/{room}`, `heartbeat/{level}/{role}`
Server → edge	MQTT	`commands/{level}/doors` {action: release | lock}; `commands/{level}/stand-down` {room, floor_clear, relock}
Edge → BuildSim	REST	`PUT /api/actuators/{id}/state` {closed_unlocked | closed_locked}
Server → BuildSim	REST	`/api/effects`, `/api/alerts`, `/api/room-appearance`, `/api/coverage`; actuators of a taken-over floor
Server → LLM	HTTP	LiteLLM chat completion; JSON response `{assessment: {is_fire, confidence, reasoning}, alert: {severity, title, message}}`
Pipeline → TimescaleDB	SQL	`sensor_readings`, `edge_decisions`, `heartbeats`, `llm_overrides`, `fire_events`
Operator → control API	REST	`POST /fire/start`, `/fire/stop`, `/false-alarm/trigger`, `/viz/clear`; `GET /status`
Grafana ← TimescaleDB	SQL	dashboards built by `grafana/build_dashboards.py`

A controller subscribes to its own floor only. Door states are BuildSim's own: closed_locked is the normal state, closed_unlocked is released. A server lock is unconditional for a floor without a fire of its own; a floor whose controller released the doors itself keeps them released until its own stand-down. A heartbeat carries node, role and whether the controller is acting; a standby heartbeat with acting=true is how the dashboard shows a promotion.

6. Simulating the sensor values (the physical model)

BuildSim provides the rooms but not their physics, so the readings come from a physics engine. It does not try to be a fire model; it produces readings with enough structure (trends, spread between rooms, a response to door and sprinkler state) that both tiers have something real to decide on and the control loop closes.

Temperature follows a heat balance per room: loss to the outside, heat from the fire in proportion to its intensity, heat from occupants, and a sprinkler term. Sprinklers are mechanical: a head activates when the room temperature passes its activation point and then reduces the fire term. Neither tier commands them.

Smoke and CO rise with fire intensity and decay exponentially once the fire is out. CO decays more slowly than smoke: after a 45 s fire the fire room's CO halves roughly every minute, which is why ALL CLEAR comes about seven minutes after the fire is stopped. A false alarm raises smoke but not CO; that difference is what both tiers use to separate a fire from smoke alone.

Propagation. Each tick, a burning room pushes heat, smoke and CO to its neighbours with a factor $1 - d/60$ of the centroid distance d. Neighbours are the rooms joined to it by an edge of BuildSim's door graph, plus any room within 30 units.

Occupancy sets the baseline heat load from per-room-type daily profiles and sampled head counts.

Sensor degradation. The gateway degrades the ground truth with four fault modes: Gaussian noise, response lag, slow drift, and stuck sensors that freeze at a value and recover at random.

7. Autonomous services and data pipeline

7.1 The autonomous services

Tier 1: the edge controller (`edge/edge_controller.py`) runs as one process per floor and role. It keeps a short window of readings per sensor per room, records when each sensor's value last changed, and every 5 s evaluates each candidate room as in Figure 3. The isolation forest runs through ONNX Runtime on six features (smoke, CO, temperature and their rates); on the Pi 400 this takes 17.8 ms p50 and 33 ms p95 per room. On the first `fire` on its floor the controller releases all of its doors with 16 concurrent PUTs, then alerts the server. When a candidate room drops back below threshold it publishes a `clear` decision, so the server's view of the room is never left on an old value. It accepts three commands: `release` and `lock` for the doors, and `stand-down` for a room, which it refuses if that room still reads critical. It re-locks only when the floor is clear and it owns the release, or when the server sends `relock` because the controller is new. The standby runs the same code on the same readings with `acting=false`.

Tier 2: the server agent (`src/fds/agent/agent.py`) consumes alerts from a queue with three in flight. Its policy is Figure 4. Two rules protect the edge's decision. A doubted alert never undoes it: the floor's doors stay released until the readings decide. And every stand-down carries what the server knows and the controller may not: whether the floor is clear and whether to re-lock. The agent also keeps the per-floor heartbeat table. A floor silent for 30 s (90 s after the server's own boot) is taken over: the server subscribes to that floor's readings, applies the threshold rule itself and actuates the doors directly. The first heartbeat from a controller hands the floor back.

Why two techniques. The isolation forest learns what normal looks like from real operation and runs on any hardware. The LLM reads context (the faulty-sensor list, spread risk, the route) and explains its verdict. Neither is trusted alone: on the edge the rule overrides the model, and

on the server the readings override the LLM. The LLM's confidence is not a calibrated probability (Section 13), so it only gates escalation and stand-down, and its rejection of an edge-confirmed room puts the room on watch rather than reversing the release.

Degradation. LLM unreachable: the fallback applies the multi-sensor rule and the strong-smoke-with-stale-CO rule with confidence 0.9 or 0.2, and the viewer cards say "fallback mode". Broker restart: the MQTT client reconnects and re-subscribes on its own; the controller keeps ticking on its last readings in the meantime and retries its initial connect with backoff. Server restart: the controllers keep their floors, and the next stand-down carries `relock`. Controller restart or node loss: Section 8.

Retraining (`fds.models.retrain`, a ClearML task) rebuilds the detector from real operation: per-room 5 s windows from the readings table, the windows the LLM rejected (weighted $\times 20$) and augmented false-alarm windows, fitted with contamination 0.02, evaluated against real normal rooms, LLM-confirmed fires, held-out false-alarm windows and the deployed model, then exported to ONNX and checked for sklearn/ONNX parity. The first retrain on real data replaced a model trained on synthetic data that had flagged 97 % of real normal rooms. The retrain after the acceptance run reproduced the deployed model (Section 11).

7.2 The data pipeline

The pipeline consumer subscribes to `building/#`, `decisions/#`, `alerts/#` and `heartbeat/#` and writes to five tables: `sensor_readings` (183 M rows, 18 GB after eight days), `edge_decisions` (every Tier-1 decision, alert and door event, with `if_ms` and `if_anomaly`), `heartbeats`, `llm_overrides` (every Tier-2 verdict, watch, stand-down, takeover and failover, with `llm_ms`) and `fire_events`. Grafana reads them directly. The retraining job reads readings and verdicts, so the LLM's judgements become the labels for the next edge model (Figure 5).

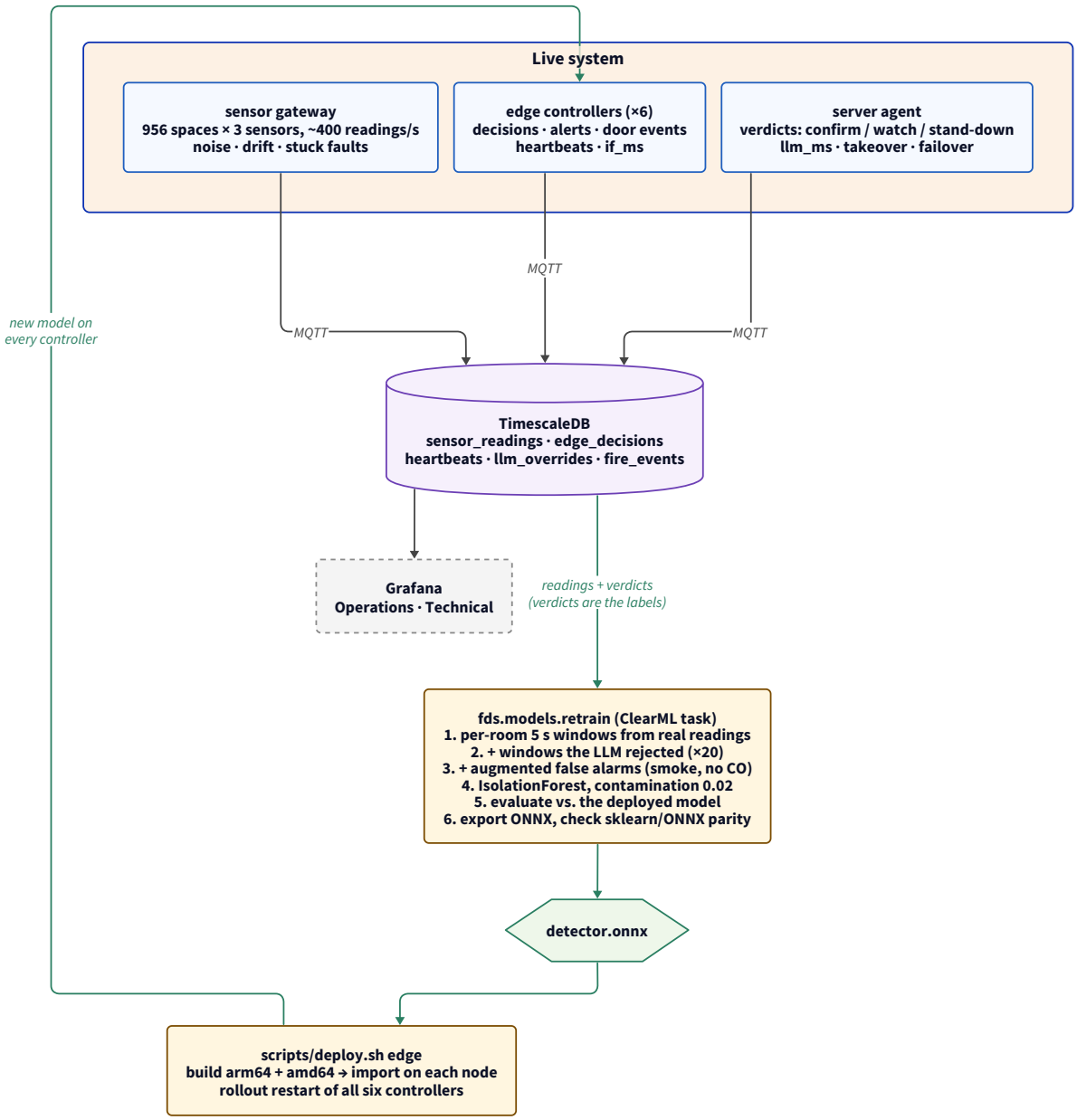


Figure 5: Data pipeline and the retraining loop: readings and verdicts flow into TimescaleDB; the retrain task turns verdicts into labels and ships a new ONNX model to every controller.

Measured ingestion is about 410 readings/s sustained, with the consumer queue at zero between bursts; a start-up burst of 974 queued messages drained within a minute.

8. Behaviour

8.1 Key scenario

Figure 6 shows the baseline fire from the acceptance run (scenario 6) with measured timestamps. The fire is started in A109 at 13:45:52.680 UTC. The level0 controller confirms at 13:46:07.357 on smoke 14.3 and CO 37.7 and releases all 41 doors on its floor in the same tick, 14.7 s after ignition, without the server. The server receives the alert 1 ms later, gets the LLM verdict after 2 967 ms, and releases level1 and level2 25 ms after that. The fire is stopped at 13:46:37.688. Rooms stand

down one by one once their readings have been clear for 60 s, and ALL CLEAR re-locks all three floors at 13:53:59.171.

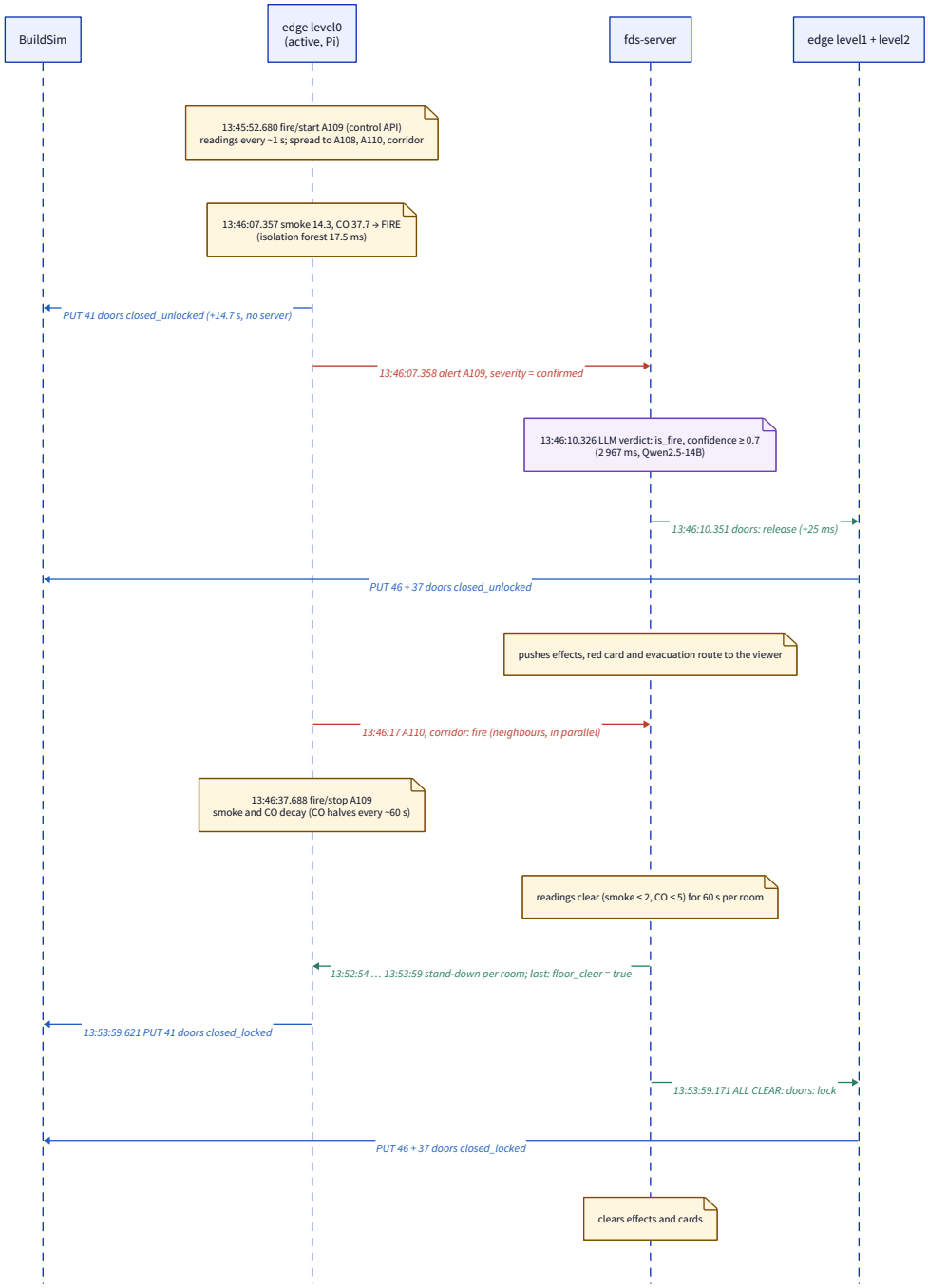


Figure 6: Baseline fire, scenario 6, with measured timestamps. The edge acts within 15 s; the server adds the building-wide response three seconds later.

Two more sequences were measured. In **failover mid-incident** (Figure 7, scenario 8) the level1 active pod was deleted while its doors were released. A standby promotes after 30 s without the active's heartbeat and demotes when the heartbeat returns; here it promoted 30 s after the last heartbeat, reported the failover, left the doors as they were, and demoted 5 s later when the replacement pod sent its first heartbeat. The replacement pod had no memory of the incident; it locked the doors at stand-down because the command carried `relock=true`. In **server takeover** (Figure 8, scenario 1) both level2 controllers were scaled to zero. The server took the floor over

30 s later, released its 37 doors directly when a level0 fire was confirmed, and handed the floor back 20 s after the controllers returned.

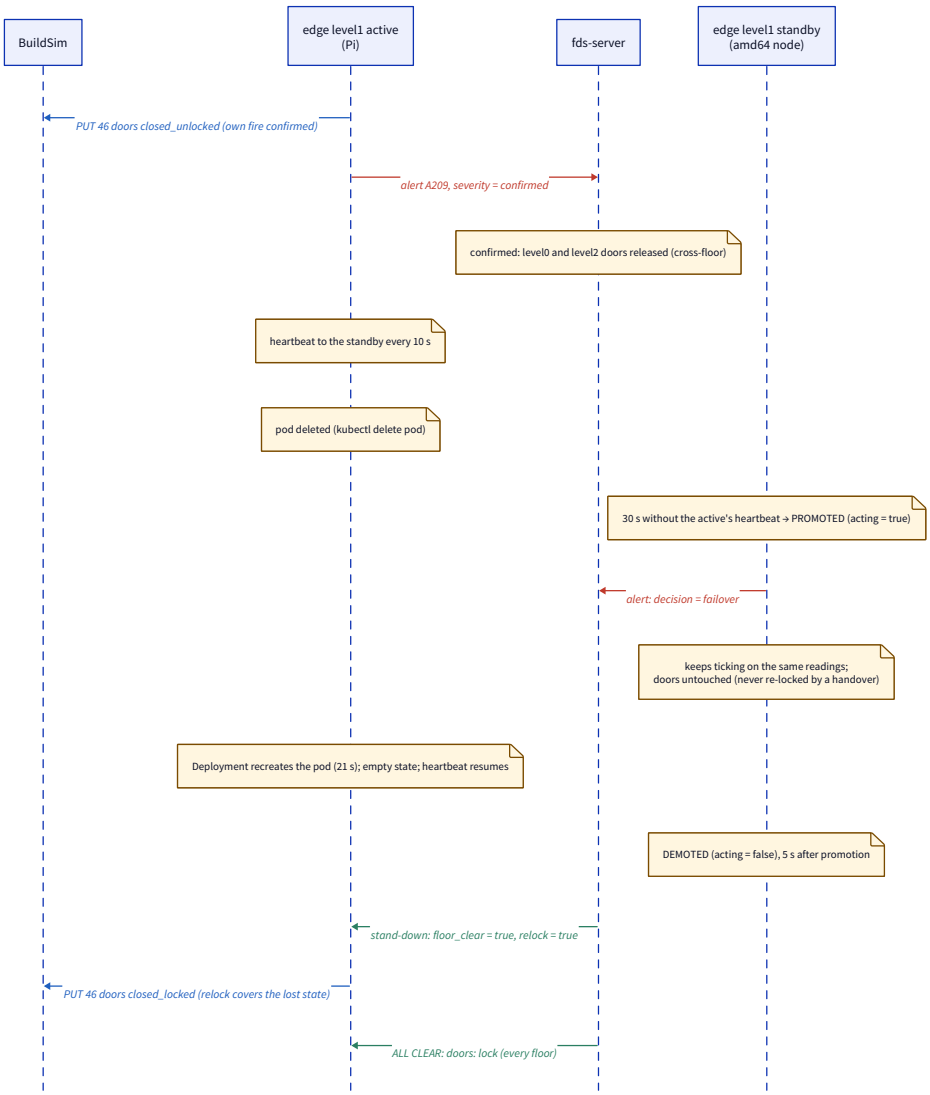


Figure 7: Scenario 8: active controller lost mid-incident.

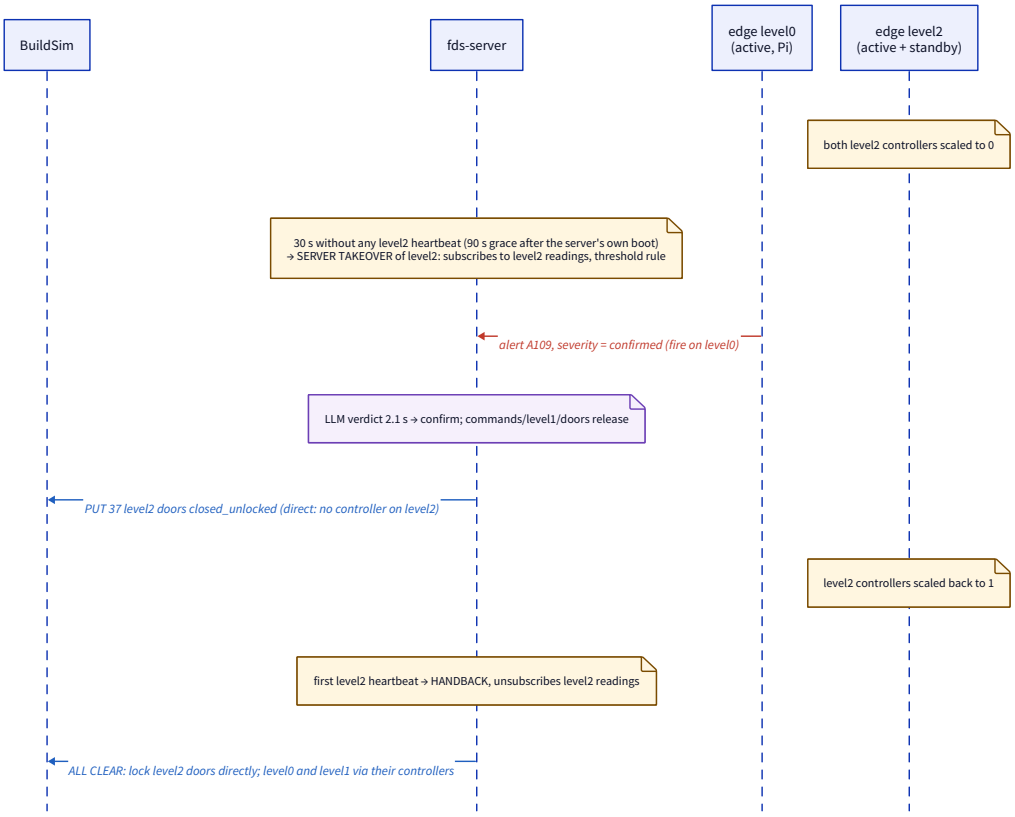


Figure 8: Scenario 1: no controller on level2; the server covers the floor.

8.2 Door lifecycle (state diagram)

The state diagram (Figure 9) is for the fire doors of one floor, since that is the state the system exists to control. Either tier may release the doors. Closing them needs more: the edge re-locks only when its floor is clear, and it refuses a stand-down while a room still reads critical. A controller that has just started does not know what it released, so every stand-down carries `floor_clear` and `relock` from the server's view, and the ALL CLEAR lock is unconditional for a floor without a fire of its own. A floor under server takeover follows the same state machine with the server in both roles.

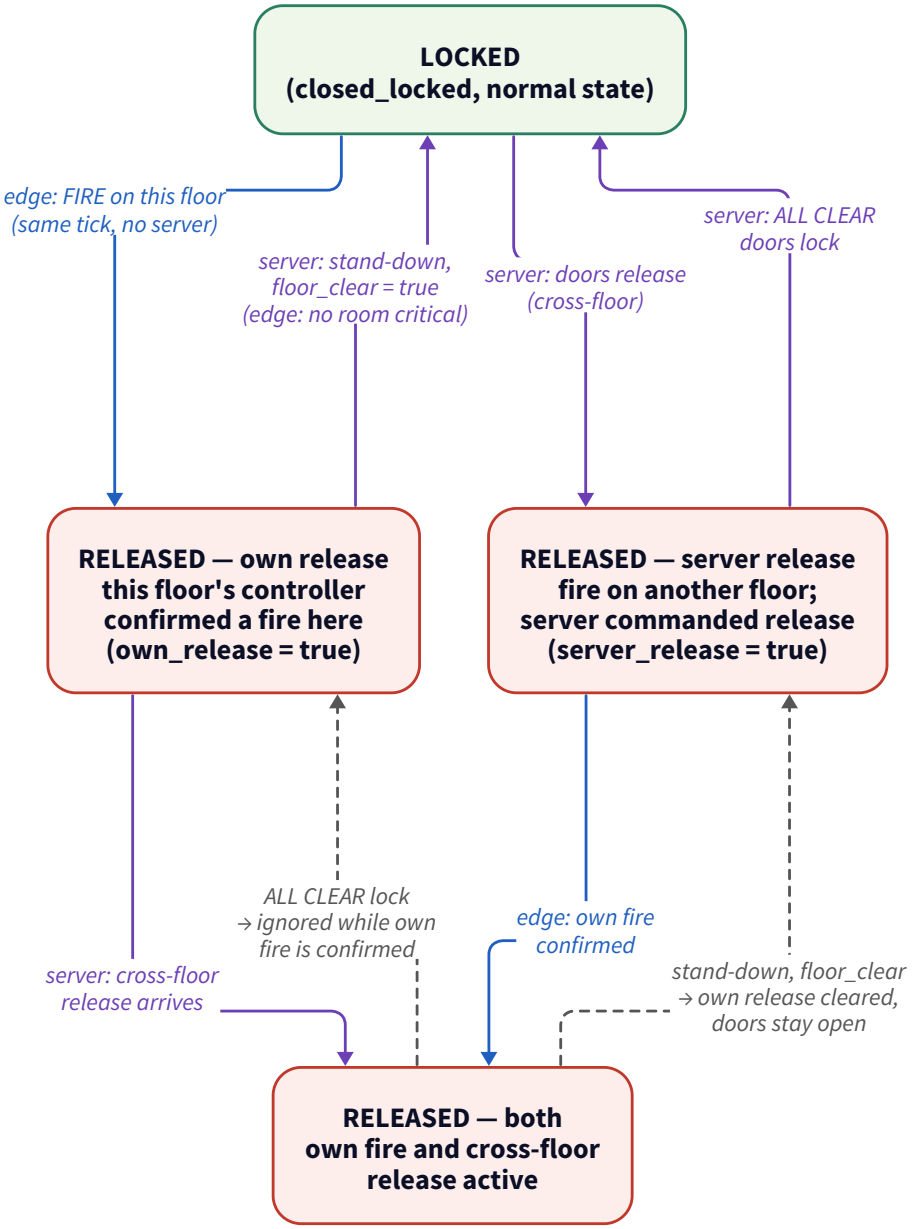


Figure 9: Door states of one floor and which tier may move between them.

Each controller also has a small lifecycle of its own: it starts, discovers its doors (retrying until BuildSim answers), connects to the broker (retrying with backoff), runs its tick, heartbeat, peer-watch and command loops, and on SIGTERM cancels them and exits 0, so stopped pods show as Completed.

9. Deployment and component placement

The system runs in namespace `ai-edge-intel-course` of a k3s cluster (Figure 2). Placement follows the two tiers: detection on the edge nodes, everything else on the server node.

Role (node)	Hardware	What runs there and why
Edge node (`pi400-master-01`)	Raspberry Pi 400, ARM64, 4 GB	The three active edge controllers (150 m CPU / 192 Mi requested each). This is the role a fire panel has in a real building: detection and door release with no dependency on the rest of the cluster.
Standby node (`infinity-magic`)	amd64 worker	The three standby controllers, same image and spec, on a different node so that one node failure cannot take both controllers of a floor.
Server node (`proart-master-01`)	Ryzen 9 9950X, 128 GB, RTX 5060 Ti 16 GB	Control plane; `fds-server` (physics, agent, pipeline, control API); BuildSim; the LLM (SGLang behind LiteLLM, namespace `ai-inference`); Mosquitto and TimescaleDB (`data-infra`); Grafana, Phoenix, ClearML, Vault.

Images are built on the control plane with `buildx` for arm64 and amd64, copied to each node and imported into containerd (imagePullPolicy: Never). `scripts/deploy.sh` does the build, copy, apply and restart. `deploy.sh check` is a pre-flight that confirms the images are present on each node, the application Secret exists and the pods are healthy. The server pod gets its secrets from the Kubernetes Secret fds-app; there is no Vault token in the namespace. After a full stop the start order is BuildSim first, then the server and the six controllers together. The server waits 90 s after boot before it will take over a floor, because starting it 45 s ahead of the controllers once made it take over all three floors.

10. Test plan

Testing has three layers. **Unit tests** run offline with the broker, BuildSim, the ONNX session, the LLM and the database replaced by in-memory stand-ins: 68 tests covering the Tier-1 controller (decision rule, alert gate, door ownership, failover, broker retry, stale sensors), the Tier-2 policy (dedupe, verdict gate, watch, stand-down, ALL CLEAR, takeover, viewer cards), the fallback with the LLM unreachable, and the physics, sensor gateway, detector, spread, evacuation and occupancy models. CI runs `ruff` and the suite on every push. **Acceptance scenarios** run against the live cluster with the viewer and the dashboards open; each has a pass criterion that is checked in the server log, the controller logs, the database and BuildSim's door states (`scripts/door_check.py`). **Data checks** (`scripts/db_check.py`) count the rows each run produced.

ID	Scenario	Requirements	Pass criterion
T-01	Scale both level2 controllers to 0; fire on level0; restore level2	SAF-03, SAF-01	Takeover logged ≈30 s after scale-down; level2 doors released directly; hand-back on first heartbeat; ALL CLEAR re-locks all floors
T-02	Scale LiteLLM to 0; fire on level1; restore	NFR-02, FR-03	Fallback confirms within one tick; cross-floor release; cards in fallback mode; verdicts resume after restore
T-03	Restart Mosquitto; fire afterwards	NFR-06, NFR-07	Edges and server reconnect; no takeover; fire handled with real verdicts
T-04	Cold start of the namespace in the documented order	NFR-01	Doors discovered on every floor; six heartbeats; no takeover; no spurious alert for 2 min; 950 rooms in `/status`
T-05	Smoke-only event (`/false-alarm/trigger`)	FR-02, SAF-04	Edge possible; LLM rejects; watch; stand-down with `relock=false`; no door commands
T-06	Baseline fire, 45 s, all systems healthy	FR-01, FR-03, FR-04, FR-05, FR-06, SAF-01, NFR-04	Edge release within 60 s of ignition; verdict; cross-floor release within 1 s; routes; spread to adjacent rooms only; ALL CLEAR

ID	Scenario	Requirements	Pass criterion
T-07	Fire on level0 and a smoke-only event on level1 within seconds	FR-02, SAF-04	Parallel verdicts; level1 on watch and never escalates on its own; its doors released only by cross-floor release and re-locked only at ALL CLEAR
T-08	Delete the active controller pod while its floor is released	SAF-02, SAF-04	Standby promotes within 30 s; doors stay released; stand-down and re-lock through whichever controller is present; demote on return
T-09	Data and model: row counts, dashboards over 3 h, retrain on the new data	NFR-03, NFR-04	All tables grew; dashboards show every scenario; retrain metrics at or above the deployed model; ONNX parity 1.000
T-10	CI, secret scan, pod spec review	NFR-05	CI passes (`ruff` and 68 tests); no secrets in the repository; no Vault token in the namespace
T-U	Unit suite	FR-05, FR-06, NFR-07 and all policy rules	68 passed

11. Evaluation and results

The system was evaluated at full building scale in a two-day acceptance run. Every number in this section comes from a log line, a database query or a test output recorded during that run.

Acceptance scenarios. All nine passed. Scenarios 1 and 2 were run twice because the first run of each found a defect, which was fixed and re-verified (see below).

T	Result	Measured
T-01	pass (2 runs)	takeover at +30 s; 37 level2 doors released directly; hand-back 20 s after the controllers returned
T-02	pass (2 runs)	fallback confirmed the origin room 84 ms after the alert; cross-floor release; cards marked “fallback mode”
T-03	pass	server reconnected in 6 s, controllers in 13 s; no takeover; fire handled with verdicts of 1.9–2.5 s
T-04	pass	318/351/287 rooms and 41/46/37 doors discovered; six heart-beats; a start-order takeover seen once, fixed with the boot grace
T-05	pass	rejected in 1.9 s (confidence 0.10); watch; stand-down at +2 min; no door events
T-06	pass	see the latency table
T-07	pass	two verdicts 1.7 s apart on different floors; level1 stayed on watch; A109 confirmed by the LLM with a dead CO sensor
T-08	pass	promote 30 s after the last heartbeat, demote 5 s later; doors held; final state all locked
T-09	pass	183 M readings / 18 GB; retrain reproduced the deployed model; parity 1.000

Latency (FR-01, FR-03, SAF-01, NFR-04). From scenario 6 and the 48-hour database window around the run:

Step	Measured	Requirement
Ignition → edge `fire` decision (A109)	14.7 s	< 60 s
Edge decision → own-floor doors released	same tick	same tick
Isolation forest per room, Pi 400	p50 17.8 ms, p95 33.0 ms (n=3 035)	p95 < 100 ms
Isolation forest per room, amd64 standby	p50 2.3 ms, p95 6.4 ms (n=3 000)	—
Edge alert → server receipt	≈1 ms	—
LLM verdict (confirm / reject, n=40)	p50 2.1 s, p95 3.2 s	—
Verdict → cross-floor release command	25 ms	< 1 s
Standby promotion after controller loss	30 s	≤ 30 s
Server takeover of a silent floor	30 s	≤ 30 s
Fire out → ALL CLEAR (45 s fire)	7 min 21 s	after 60 s of clear readings (FR-06)

Verdict latency fell from 5.2 s p50 / 7.0 s p95 with the original full-plan prompt to 2.1 / 3.2 s with the verdict-only contract. Queueing between alert and verdict under a cluster of alerts fell from 24 s p50 to none once alerts were assessed three at a time.

Detection model (FR-02, NFR-04). The edge model is an isolation forest trained on real operation. The retrain at the end of the acceptance run used 16 000 real normal windows, 36 LLM-confirmed fire windows, 16 LLM-rejected windows and 1 600 augmented false-alarm windows:

Metric	Deployed model	Retrained model
Real normal rooms not flagged	0.998	0.997
LLM-confirmed fires detected	0.917 (33/36)	0.917
sklearn / ONNX parity	0.999	1.000

The deployed model was kept. The model the project started with, trained on synthetic data, flagged 97 % of real normal rooms at building scale; training on real windows is what made the ambiguous band usable. The three fires the model misses are caught by the deterministic rule, which is why the rule is evaluated first and the model cannot override it.

Scale (NFR-01, NFR-03). All 950 rooms (956 published spaces) are monitored, and each controller reports its floor's room count every tick. Ingestion sustained about 410 readings/s with the consumer queue at zero; a start-up burst of 974 messages drained within a minute. Eight days of operation produced 183 M readings (18 GB), 252 k heartbeats, 11 k edge decisions and 417 Tier-2 events.

Defects found during the run. Six, all fixed, re-tested and committed during the run:

1. **Spread across a courtyard.** A 50-unit centroid radius made A323 a neighbour of A109, so every A109 fire appeared in the far wing. Neighbours are now door-graph edges plus a 30-unit wall radius.
2. **A stuck sensor held the stand-down.** `level0-A108-co` froze at 5.09 ppm for four minutes, just above the 5.0 clear threshold, and kept three floors open until it recovered. The edge now reports a sensor frozen for 60 s as stale, and the stand-down ignores it (never both smoke and CO at once).
3. **The fallback doubted the origin room.** With the LLM down, a reading-scaled confidence put a clear fire at 0.25; the neighbours were confirmed first and the cross-floor release went out 25 s late, on the wrong room. The fallback now applies the multi-sensor rule with confidence 0.9.
4. **A dead CO sensor in the fire room blocked detection.** `level0-A109-co` was stuck at 0.00, so the two-sensor rule could never fire, and the LLM read "smoke without CO" as a false alarm; the floor was handled 11 s later through the neighbours. Both tiers now use the stale list: strong smoke alone confirms when CO is stale, and the LLM is told which sensors are faulty.

5. **Start-order takeover.** Starting the server 45 s before the controllers made it take over all three floors; a 90 s boot grace fixed it.
6. **Duplicate door writes.** The stand-down re-lock and ALL CLEAR both locked the same doors; a controller now skips a state it has already written.

Limitations of the results. The LLM's confidence is not a calibrated probability. In scenario 8 it rejected an edge-confirmed room at confidence 0.00 on borderline readings, and the room was escalated a minute later by the watch rule on readings alone. The design tolerates this, but it means the 0.7 gate is a tuning constant rather than a probability. The 14.7 s from ignition to release is set by the physics ramp and the 5 s tick, not by inference; a 1 s tick would cost CPU on the Pi for little gain. The seven-minute ALL CLEAR follows from the CO decay model and is correct, but it is slow to watch. Finally, the system was tested against its own simulator. The fault models are plausible, but they have not been validated against real sensors.

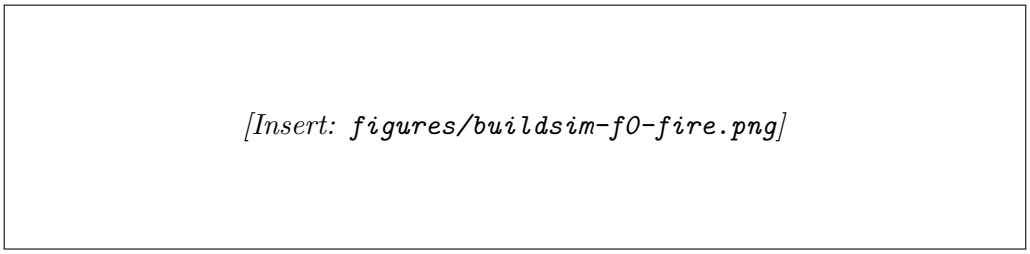


Figure 10: BuildSim viewer during scenario 6: fire effects and cards on A109 and its neighbours, doors released.



Figure 11: Operations dashboard, 3-hour window covering the acceptance scenarios: controllers, door state, incident timeline per room, readings of the rooms Tier 1 flagged, Tier-1 and Tier-2 event rates.

12. Dashboard

Two Grafana dashboards are generated from code (`grafana/build_dashboards.py`) and provisioned by script.

Operations dashboard, for the operator:

- A stat row: edge controllers alive (of 6), confirmed fires, rooms on watch, floors with doors released, floors under server takeover, and sensor feed in rooms per minute. All green with empty charts below means the building is operating normally.
- An edge-controller table: level, role, node, alive, acting. A promoted standby shows `ACTING`.
- Door state per floor with the time of the last change.
- An incident timeline with one row per room that has had a Tier-2 event, coloured by state (confirmed, watching, escalated, stood down).
- Smoke, CO and temperature of the rooms Tier 1 has flagged, so the charts follow the incident rather than a fixed room.
- Tier-1 decisions and Tier-2 verdicts per 30 s, the incident log with reasons, and the edge alert table.

Technical dashboard, for engineers: isolation-forest latency per node, alert-to-verdict latency, LLM inference p50/p95 and per call, heartbeats per controller, failover and takeover events, verdict mix, pipeline throughput and table sizes.

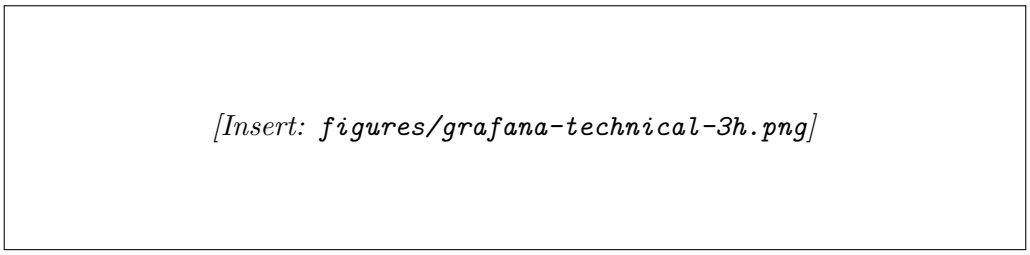


Figure 12: Technical dashboard, 3-hour window covering the acceptance scenarios.

The BuildSim viewer is the third display, and the one that shows the behaviour to someone who is not an engineer: fire effects, red cards for confirmed rooms, orange "under observation" cards for rooms the LLM doubted, door states, and everything cleared at ALL CLEAR.

13. Risks and critical reflection

A server restart in the middle of an incident loses the incident state. The controllers keep their floors released, so nothing is lost on the life-safety side, but the new server process does not know an incident is open and will never send the stand-down. This did not occur during the acceptance run, because each redeploy happened after ALL CLEAR. The mitigation is designed but not built: an edge-side self-clear after a long clear window, and a door-state recovery when the server boots. This is an important known limitation.

The LLM's confidence is a number, not a probability. It works as a gate because the readings decide in the end, but it is the weakest part of the decision path, and the acceptance run shows how little it adds there: in nine scenarios its verdicts never changed an outcome that the rule-based fallback would not have produced, and the one time it disagreed with the edge (scenario 8, confidence 0.00) it was wrong and the readings overruled it a minute later. What the LLM does contribute is the reasoning text and the alert wording, which the operator and the incident log use. The verdict itself needs a calibrated classifier with the right inputs (readings, rates, the stale list, spread risk, route), not a reasoning model. Decision models of this kind (Jev, and open-weight equivalents such as OpenJev and Laya) became available during the project; they return typed answers with probabilities in a few hundred milliseconds. The architecture is model-agnostic at the verdict boundary, so the next step is a Tier 2 where a calibrated decision model gives the verdict and the LLM, off the decision path, writes the explanation. The evaluation would replay the acceptance run's alerts through such a model and compare verdicts and latency with the Qwen results stored in `llm_overrides`. Such a model belongs in Tier 2, not on the edge: the isolation forest answers a different question (is this room unlike this building's normal?) at a cost a Pi can afford for every room every tick, and the rule, not a learned model, decides fire on the life-safety path.

Thresholds are hand-set. Smoke 10 and CO 20 for fire, smoke 5 for the ambiguous band, smoke 2 and CO 5 for clear, 60 s windows, confidence 0.7. They were chosen from the physics model and adjusted during the acceptance run. A real installation would need them calibrated per sensor type and per building.

The simulator sets the difficulty of the problem. The physics engine and the sensor faults were written for this project. Two assumptions matter most. The fault models (noise, lag, drift, stuck) are reasonable but were not derived from real sensor data. And the physics makes smoke

and CO separate a fire from a false alarm more cleanly than real sensors would; a real building has ventilation, open windows and cooking, none of which the model includes. The results show the requirements are met in the simulated building. Whether the same thresholds and timings hold in a real one would have to be established with real sensor data.

Risk / assumption	**Impact**	**Mitigation / decision**
LLM unavailable	No contextual judgement	Rule-based fallback, exercised live (T-02)
Active controller dies	Floor undetected	Standby on another node, 30 s (T-08)
Both controllers of a floor gone	Floor undetected	Server takeover with direct door actuation (T-01)
Server restarts mid-incident	Stand-down never sent	Known limitation; edge self-clear and boot recovery designed
Stuck sensor	Detection blocked or stand-down held	Stale-sensor rule in both tiers (T-03, T-U)
Broker restart	Messages lost for seconds	Automatic reconnect; edge keeps ticking (T-03)
LLM confidence uncalibrated	Wrong gate decisions	Readings decide on watch; hybrid Tier 2 as future work
Edge image garbage-collected on the Pi	Actives cannot start	`deploy.sh check` before every start
Simulated physics optimistic	Detection times differ in a real building	Results reported for the simulated building only
Cross-floor release opens every door on the other floors	More doors released than a real building would	Accepted for this project; a stairwell-aware release along the affected routes is the design change at larger scale

Use of AI tools

AI coding assistants were used during this project. The architecture, the design decisions in Section 4.4, the requirements, the test plan and the acceptance scenarios are my own, and every decision in the report was taken and verified by me on the running system. The assistants were used to write and refactor code from my specifications, to generate the first drafts of unit tests and diagram sources, and to help find the cause of defects during the acceptance run. All generated code was reviewed, run and tested by me before it was committed, and the results reported in Section 11 were measured by me on the cluster.